C programming---basic

1 Introduction to C
2 C Fundamentals
3 Formatted Input/Output
4 Expression
5 Selection Statement
6 Loops
7 Basic Types
8 Arrays
9 Functions
10 Pointers
11 Pointers and Arrays

Introduction to C

Intended use and underlying philosophy

1 C is a low-level language

---suitable language for systems programming

2 C is a small language

---relies on a “library” of standard functions

3 C is a permissive language

---it assumes that you know what you’re doing, so it allows you a wider degree of latitude than many languages. It doesn’t mandate the detailed error-checking found in other language

Introduction to C

Strengths:

+ Efficiency: intended for applications where assembly language had traditionally been used.

+ Portability: hasn't splintered into incompatible dialects; small and easily written

+ Power: large collection of data types and operators

+ Flexibility: not only for system but also for embedded system commercial data processing

+ Standard library

+ Integration with UNIX

Introduction to C

Weaknesses:

+ error-prone

+ difficult to understand

+ difficult to modify

Similarities of C to java

- /* Comments */
- Variable declarations
- if / else statements
- for loops
- while loops
- function definitions (like methods)
- Main function starts program

Differences between C and java

- C does not have objects
  There are “struct”ures
- C is a functional programming language
- C allows pointer manipulation
- Input / Output with C
  Output with **printf** function
  Input with **scanf** function

C Fundamentals

First program

```
#include <stdio.h>
main()
{
    printf(“To C, or not to C: that is the question”);
}
```

C Fundamentals

Compiling and Linking

Preprocessing: the program is given to a preprocessor, which obeys commands that begin with **#**(**directives**) add things to the program and make modifications

Compiling: modified program→compiler→object code

Linking: add library functions to yield a complete executable program

C Fundamentals

Compiler

```
% cc –o pun pun.c
% gcc –Wall –o pun pun.c
```

C Fundamentals

Keywords

auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
const	float	short	unsigned
continue	for	signed	void
default	goto	sizeof	volatile
do	if	static	while

Variable Type

C has the following simple data types:

C type	Size (bytes)	Lower bound	Upper bound
char	1	—	—
unsigned char	1	0	255
short int	2	-32768	$+32767$
unsigned short int	2	0	65536
(long) int	4	-2^{31}	$+2^{31}-1$
float	4	$-3.2 \times 10^{\pm 38}$	$+3.2 \times 10^{\pm 38}$
double	8	$-1.7 \times 10^{\pm 308}$	$+1.7 \times 10^{\pm 308}$

Variable Type

Java has the following simple data types:

Primitive type	Size	Minimum	Maximum	Wrapper type
boolean	1-bit	–	–	**Boolean**
char	16-bit	Unicode 0	Unicode 2^{16}- 1	**Character**
byte	8-bit	-128	+127	**Byte**[1]
short	16-bit	-2^{15}	$+2^{15} - 1$	**Short**[1]
int	32-bit	-2^{31}	$+2^{31} - 1$	**Integer**
long	64-bit	-2^{63}	$+2^{63} - 1$	**Long**
float	32-bit	IEEE754	IEEE754	**Float**
double	64-bit	IEEE754	IEEE754	**Double**

Basic Types

Type (16 bit)	Smallest Value	Largest Value
short int	-32,768(-2^{15})	32,767(2^{15}-1)
unsigned short int	0	65,535(2^{16}-1)
Int	-32,768	32,767
unsigned int	0	65,535
long int	-2,147,483,648(-2^{31})	2,147,483,648(2^{31}-1)
unsigned long int	0	4,294,967,295

Basic Types

Type (32 bit)	Smallest Value	Largest Value
short int	-32,768(-2^{15})	32,767(2^{15}-1)
unsigned short int	0	65,535(2^{16}-1)
Int	-2,147,483,648(-2^{31})	2,147,483,648(2^{31}-1)
unsigned int	0	4,294,967,295
long int	-2,147,483,648(-2^{31})	2,147,483,648(2^{31}-1)
unsigned long int	0	4,294,967,295

Data Types

- char, int, float, double
- long int (long), short int (short), long double
- signed char, signed int
- unsigned char, unsigned int
- 1234L is long integer
- 1234 is integer
- 12.34 is float
- 12.34L is long float

Reading and Writing Integers

```
unsigned int u;
scanf("%u", &u);     /* reads u in base 10 */
printf("%u", u);      /* writes u in base 10 */
scanf("%o", &u);      /* reads u in base 8 */
printf("%o", u);      /* writes u in base 8 */
scanf("%x", &u);      /* reads u in base 16 */
printf("%x", u);      /* writes u in base 16*/

short int x;
scanf("%hd", &x);
printf("%hd", x);

long int x;
scanf("%ld", &x);
printf("%ld", x);
```

Floating Types

float	single-precision floating-point
double	double-precision floating-point
long double	extended-precision floating-point

Type	Smallest Positive Value	Largest Value	Precision
float	$1.17*10^{-38}$	$3.40*10^{38}$	6 digits
double	$2.22*10^{-308}$	$1.79*10^{308}$	15 digits

```
double x;
scanf("%lf", &x);
printf("%lf", x);
```

```
long double x;
scanf("%Lf", &x);
printf("%Lf", x);
```

Character Types

```
char ch;
int i;
i = ‘a’;                    /* i is now 97 */
ch = 65;                    /* ch is now ‘A’ */
ch = ch + 1;                /* ch is now ‘B’ */
ch++;                       /* ch is now ‘C’ */

if(‘a’ <= ch && ch <= ‘z’)

for(ch = ‘A’; ch <= ‘Z’; ch++)
```

Char Type

‘a‘, ‘\t’, ‘\n’, ‘\0’, etc. are character constants

strings: character arrays

- (see <string.h> for string functions)
- "I am a string"
- always null (‘\0’) terminated.
- 'x' is different from "x"

Type Conversion

narrower types are converted into wider types

- f + i int i converted to characters <---> integers

<ctype.h> library contains conversion functions, e.g:

- tolower(c) isdigit(c) etc.

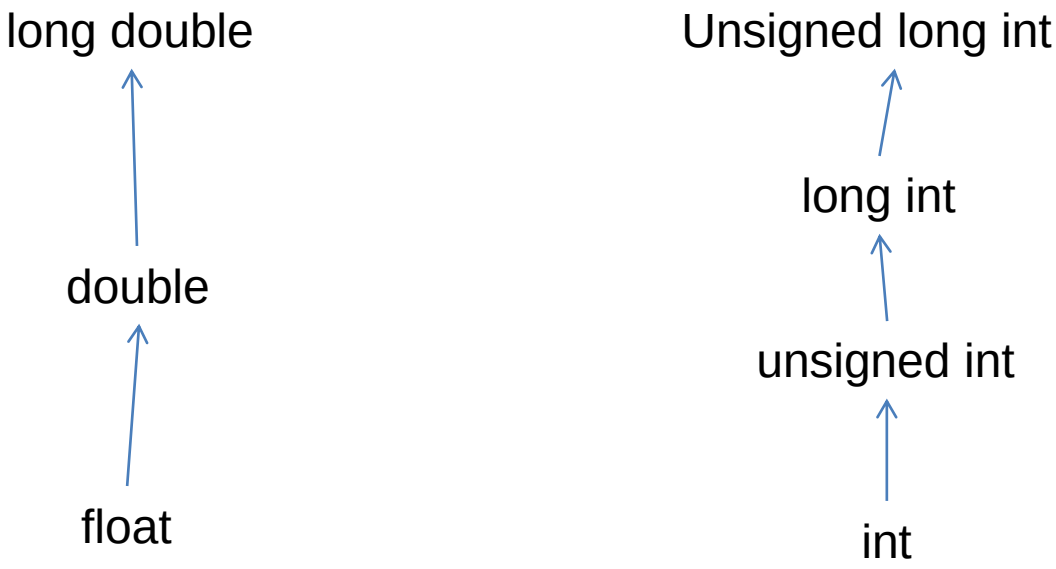
Boolean values:

- true : >= 1 false: 0

```
int atoi(char s[]) {
  int i, n=0;
  for (i=0; s[i] >= '0'
&& s[i] <= '9'; i++)
    n = 10*n + (s[i]-'0');

  return n;
}
```

Type Conversion



Type Conversion

```
char c;
short int s;
int i;
unsigned int u;
long int l;
unsigned long int ul;
float f;
double d;
long double ld;
i = i + c;   /* c is converted to int  */
i = i + s;   /* s is converted to int  */
u = u +i;  /* i is converted to unsigned int  */
l = l + u;  /* u is converted to long int  */
ul =ul + l; /* l is converted to unsigned long int  */
f = f + ul;  /* ul is converted to float  */
d = d + f;  /* f is converted to double  */
ld = ld + d;  /* d is converted to long double  */
```

Casting

(type-name) expression

```
float f, frac_part;
frac_part = f – (int) f;

float quotient;
int dividend, divisor;
quotient = (float) dividend / divisor;

short int i;
int j = 1000;
i = j * j;   /* WRONG */
```

Type Definitions

```
typedef int BOOL
BOOL flag;   /*  same as int flag; */

typedef short int Int16
typedef long int Int32
typedef unsigned char Byte

typedef struct {int age; char *name} person;
person people;
```

Formatted Input/Output

printf function

printf(string, expr1, expr2, ……..)

string: ordinary characters and conversion specifications (%)

%d --- int %s --- string %f --- float

printf(“i=%d, j=%d. x=%f\n”, i, j, x);

Formatted Input/Output

Conversion Specification

%[-]m.pX

m: specifies the minimum number of characters to print.
%4d-- _123; %-4--123_

p: depends on the choice of **X**

X:

-d: decimal form
-e: floating-point number in exponential format
-f: floating-point number in “fixed decimal” format
-g: either exponential format or fixed decimal format, depending on the number’s size

Formatted Input/Output

```
main()
{
   int i = 40;
   float x = 839.21;
   printf(“|%d|%5d|%-5d|%5.3d|\n”, i, i, i, i);
   printf(“|%10.3f|%10.3e|%-10g|\n”, x, x, x);
}
```

Formatted Input/Output

Escape Sequence

Enable strings to contain characters that would otherwise cause problems for the compiler

alert	\a	new line	\n	\”	“
backspace	\b	horizontal tab	\t	\\	\

Formatted Input/Output

How **scanf** works: is controlled by the **conversion specification**
In the format string starting from left to right.
When called, it tries to locate an item of the appropriate type
In the input data, skipping ***white-space characters***(the space,
Horizontal and vertical tab, form-feed, and new-line character)

```
scanf("%d%d%f%f", &i, &j, &x, &y);
input:
___1
-20___.3
___-4.0e3

___1*-20___.3*___-4.0e3*
sss r s rrr sss  rrs sss rrrrrr
```

Ordinary Characters in Format String

White-space characters: one white-space character in the format string will match any number of white-space character in the input.

Other characters: when it encounters a non-white-space character in a format string, scanf compares it with the next input character. If the two characters match, scanf discards the input character and continues processing the format string. Otherwise, scanf puts the offending character back into the input, then aborts without futher processing.

%d/%d will match _5/_96, but not _5_/_96
%d_/%d will match _5_/_96

Expressions

Arithmetic operator: +, -, *, /, %, ++, --……

Relational operator: <, >, <=, >=, !=

Logical operator: &&, ||

Operator Precedence and Associativity

highest: + - (unary)
* / %
lowest: + - (binary)

-i * -j = (-i) * (-j)
+i + j / k = (+i) + (j / k)

left/right associative: it groups from left/right to right/left

The binary arithmetic operators (*, /, %, + and -) are all left associative
i – j – k = (i – j) – k i * j / k = (i * j) / k

The unary arithmetic operators(+ and -) are both right associative
- + i = - (+i)

Expression Evaluation

Precedence	Name	Symbol(s)	Associativity
1	X++/X--		left
2	++X/--X unary +/-		right
3	multiplicative	*, /, %	left
4	additive	+, -	left
5	assignment	=, *=, /=, +=, -=	right

Expression Evaluation

a = b += c++ - d + --e / -f

a = b += (c++) - d + --e / -f

a = b += (c++) - d + (--e) / -f

a = b += (c++) - d + (--e) / (-f)

a = b += (c++) - d + ((--e) / (-f))

a = b += ((c++) – d) + ((--e) / (-f))

a = b += (((c++) – d) + ((--e) / (-f)))

a = (b += (((c++) – d) + ((--e) / (-f))))

(a = (b += (((c++) – d) + ((--e) / (-f)))))

Bitwise Operations

- Applied to char, int, short, long
  - And &
  - Or |
  - Exclusive Or ^
  - Left-shift <<
  - Right-shift >>
  - one's complement ~

Example: Bit Count

```
/*
   count the 1 bits in a number
   e.g. bitcount(0x45) (01000101 binary) returns 3
*/

int bitcount (unsigned int x) {
   int b;

   for (b=0; x != 0; x = x >> 1)
      if (x & 01)     /* octal 1 = 000000001 */
          b++;

   return b;
}
```

Conditional Expressions

- Conditional expressions
- expr1? expr2:expr3;
- if expr1 is true then expr2 else expr3

```
for (i=0; i<n; i++)
   printf("%6d %c",a[i],(i%10==9||i==(n-1))?'\n':' ');
```

Control Flow

- blocks: { ... }
- if (**expr**) **stmt**;
- if (**expr**) **stmt1** else **stmt2**;
- switch (**expr**) {case ... default }
- while (**expr**) stmt;
- for (**expr1**;**expr2**;**expr3**) **stmt**;
- do **stmt** while **expr**;
- break; continue (only for loops);
- goto label;

Scope Rules

- Automatic/Local Variables
  - Declared at the beginning of functions
  - Scope is the function body
- External/Global Variables
  - Declared outside functions
  - Scope is from the point where they are declared until end of file (unless prefixed by extern)

Scope Rules

- Variables can be declared within blocks too
  - scope is until end of the block

```
{
    int block_variable;
}
block_variable = 9; (wrong)
```

Scope Rules

- Static Variables: use static prefix on functions and variable declarations to limit scope
  - static prefix on external variables will limit scope to the rest of the source file (not accessible in other files)
  - static prefix on functions will make them invisible to other files
  - static prefix on internal variables will create permanent private storage; retained even upon function exit

Hello, World

```
#include <stdio.h>
/* Standard I/O library */

/* Function main with no arguments */
int main () {
  /* call to printf function */
  printf("Hello, World!\n");

  /* return SUCCESS = 1 */
  return 1;
}

% gcc -o hello hello.c
% hello
Hello, World!
%
```

Celsius vs Fahrenheit table (in steps of 20F)

- C = (5/9)*(F-32);

```
#include <stdio.h>
int main() {
  int fahr, celsius, lower, upper, step;
  lower = 0;
  upper = 300;
  step = 20;
  fahr = lower;
  while (fahr <= upper) {
    celsius = 5 * (fahr - 32) / 9;
    printf("%d\t%d\n",fahr, celsius);
    fahr += step;
  }
  return 1;
}
```

Celsius vs Fahrenheit table Remarks

- 5/9 = 0
- Primitive data types: int, float, char, short, long, double
- Integer arithmetic: 0F = 17C instead of 17.8C
- %d, %3d, %6d etc for formatting integers
- \n newline
- \t tab

New Version Using Float

```
#include <stdio.h>
int main() {
  float fahr, celsius;
  int lower, upper, step;
  lower = 0;
  upper = 300;
  step = 20;
  fahr = lower;
  while (fahr <= upper) {
    celsius = (5.0 / 9.0) * (fahr - 32.0);
    printf("%3.0f %6.1f \n", fahr, celsius);
    fahr += step;
  }
  return 1;
}
```

New Version Using Float Remarks

- %6.2f 6 wide; 2 after decimal
- 5.0/9.0 = 0.555556
- Float has 32 bits
- Double has 64 bits
- Long Double has 80 to 128 bits
  - Depends on computer

Version 3 with “for” loop

```
#include <stdio.h>

int main() {
  int fahr;

  for (fahr=0; fahr <= 300; fahr += 20)
    printf("%3d %6.1f \n", fahr,
           (5.0 / 9.0) * (fahr - 32.0));

  return 1;
}
```

Version 4 with Symbolic Constants

```
#include <stdio.h>

#define LOWER 0
#define UPPER 300
#define STEP 20

int main() {
  int fahr;

  for (fahr=LOWER; fahr <= UPPER; fahr += STEP)
    printf("%3d %6.1f \n", fahr,
           (5.0 / 9.0) * (fahr - 32.0));

  return 1;
}
```

Character I/O

- c = getchar();
- putchar(c);

Coyp file

```
#include <stdio.h>

  int main() {
    char c;

    c = getchar();
    while (c != EOF) {
      putchar(c);
      c = getchar();
    }

    return 0;
  }
```

File Copying (Simpler Version)

- c= getchar() != 0 is equivalent to
  c = (getchar() != EOF)
- Results in c value of 0 (false) or 1 (true)

```
#include <stdio.h>
int main() {
  int c;

  c = getchar();
  while ((c = getchar())!= EOF)
    putchar(c);

  return 0;
}
```

Counting Characters

- Remarks: nc++, ++nc, --nc, nc--
- %ld for long integer

```
#include <stdio.h>
int main () {
  long nc = 0;
  while (getchar() != EOF)
    nc++;
  printf("%ld\n",nc);
}
```

```
#include <stdio.h>
int main () {
  long nc;
  for (nc=0;getchar() !=
EOF;nc++);
  printf("%ld\n",nc);
}
```

Counting Lines

```
#include <stdio.h>

int main () {
  int c, nl=0;

  while ((c = getchar()) != ‘Z’)
    if (c == '\n')
      nl++;

  printf("%d\n",nl);
}
```

Counting Words

```
#include <stdio.h>
#define IN 1
#define OUT 0
int main () {
  int c, nl, nw, nc, state;

  state = OUT;
  nl = nw = nc = 0;
  while ((c = getchar()) != 'Z') {
    ++nc;
    if (c == '\n')
      nl++;
    if (c == ' ' || c == '\n' || c == '\t')
      state = OUT;
    else if (state == OUT) {
           state = IN;
           ++nw;
         }
  }
  printf("%d %d %d\n",nc, nw, nl);  }
```

Notes about Word Count

- Short-circuit evaluation of || and &&
- nw++ at the beginning of a word
- use state variable to indicate inside or outside a word